דוח פרויקט רשתות תקשורת מחשבים - חלק 2: מערכת צ'אט

מגישים: אביטל יוחננוב, נאור אנידג'ר

בחלק זה של הפרויקט, מימשנו מערכת צ'אט מרובת משתמשים (Multi-Client Chat System) הפועלת על גבי פרוטוקול TCP. המערכת מבוססת על ארכיטקטורת Client-Server, שבה שרת מרכזי מנהל את החיבורים והלקוחות יכולים לשלוח הודעות פרטיות זה לזה.

ארכיטקטורת הקוד:

- **השרת (server.py):** מאזין בפורט 5000 לכל ממשקי הרשת (0.0.0.0). הוא משתמש ב-Multi-Threading (יצירת Thread נפרד לכל לקוח) כדי לטפל במספר משתמשים במקביל ללא חסימות. הניהול מתבצע באמצעות dictionaries (users, peers) המסונכרנים ע"י מנעול (Lock).
- **הלקוח (gui_client.py / client.py):** מתחבר לשרת ב-TCP. גם הלקוח פועל בשני threads: אחד לממשק המשתמש (שליחת הודעות) ואחד להאזנה להודעות נכנסות מהשרת בזמן אמת.
- **פרוטוקול:** הגדרנו פרוטוקול טקסטואלי פשוט (CONNECT, MSG, DISCONNECT) המופרד בירידות שורה.

הוראות התקנה והרצה:

1. יש לוודא שמותקן Python 3.6 ומעלה.
2. **הפעלת השרת:** פתחו טרמינל והריצו: python server.py.
3. **הפעלת לקוח:** פתחו טרמינל נפרד והריצו python gui_client.py (לממשק גרפי) או client.py (לממשק טקסט).
4. יש להזין שם משתמש ולהתחבר.

דוגמאות קלט ופלט:

להלן דוגמא לתרחיש הרצה של המערכת:

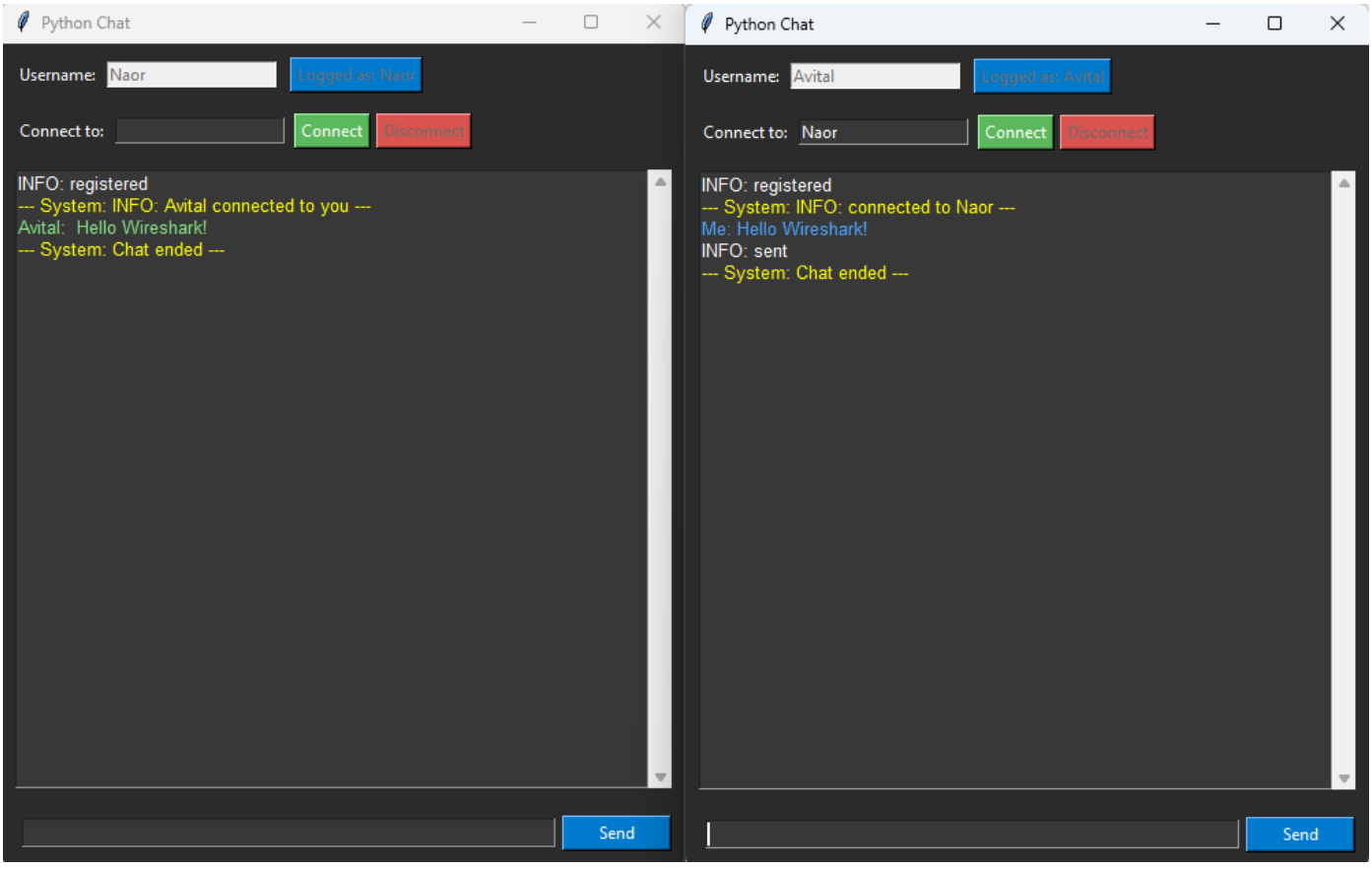
1. מסך השרת (Server Console):

בצילום המסך ניתן לראות את השרת מאזין לפורט ואת הלוגים המעידים על התחברות משתמשים (Avital, Naor) ורישומם במערכת.

```
PS C:\Users\Avital\Desktop\סימסטר ב תשפ\תקשורת תוכנה מבוססת\project part 2> python server.py
Server is running... waiting for clients
Client connected: ('127.0.0.1', 51677)
User registered: Avital
Client connected: ('127.0.0.1', 56996)
User registered: Naor
```

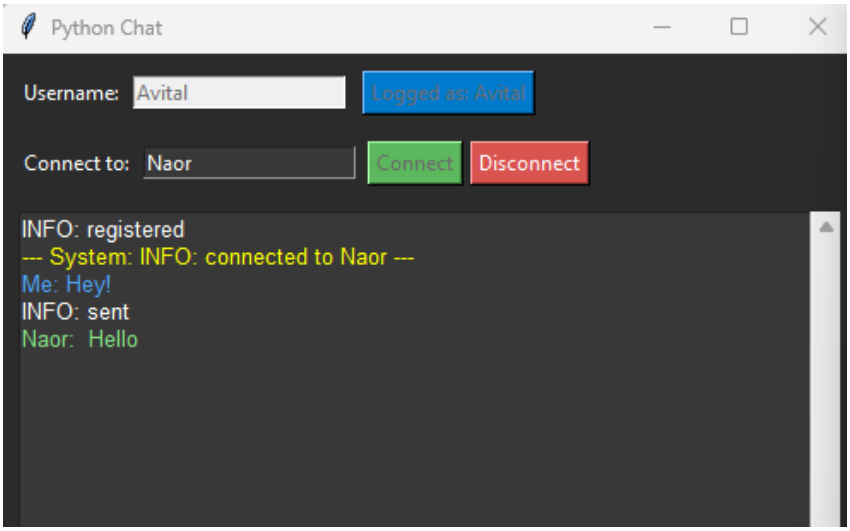
בתמונות הבאות ניתן לראות את ממשק ה-GUI שפיתחנו. המשתמשים מחוברים, ומבצעים את הפקודות הבאות:

- **קלט:** חיבור למשתמש (CONNECT).
- **פלט:** הודעת מערכת "INFO: connected to...".
- **קלט:** שליחת הודעה "Hello Wireshark!".
- **פלט:** ההודעה מופיעה בצד אחד בכחול ובצד השני בירוק.

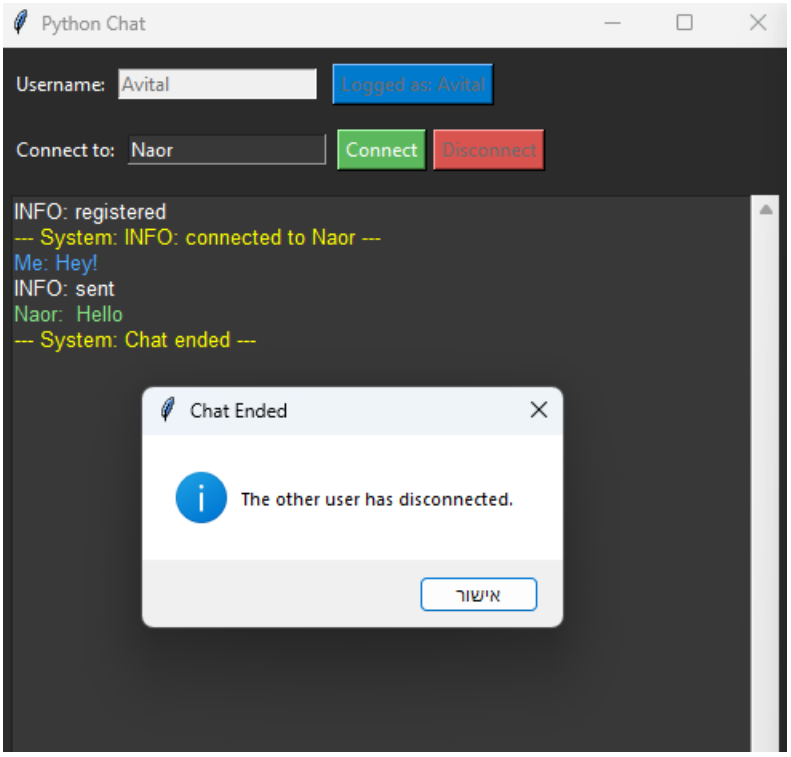


להלן עוד מספר תרחישים במערכת:

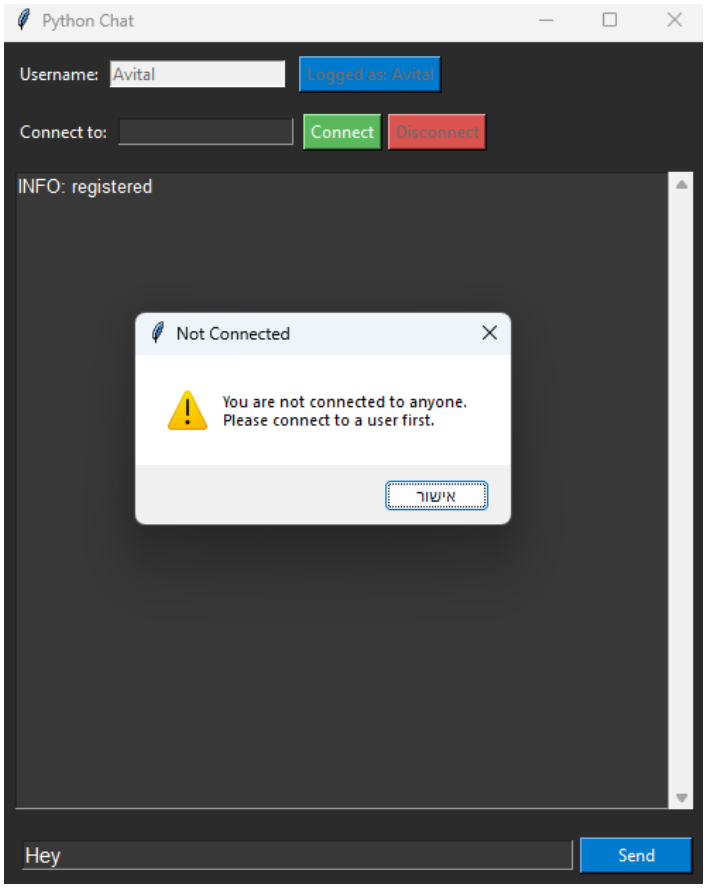
שליחת הודעה תקינה:



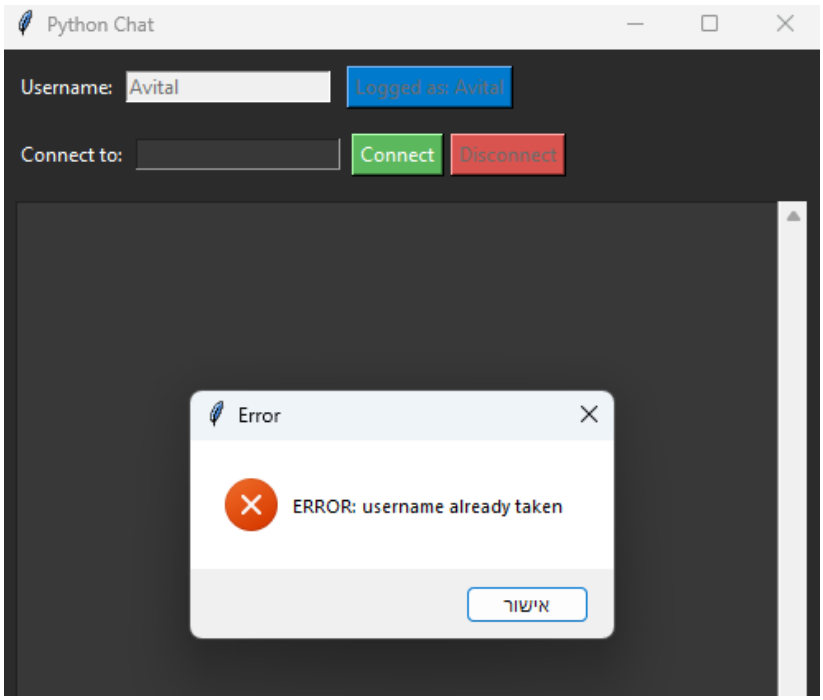
שגיאת "משתמש לא מחובר":



שגיאת "שליחה ללא יעד":

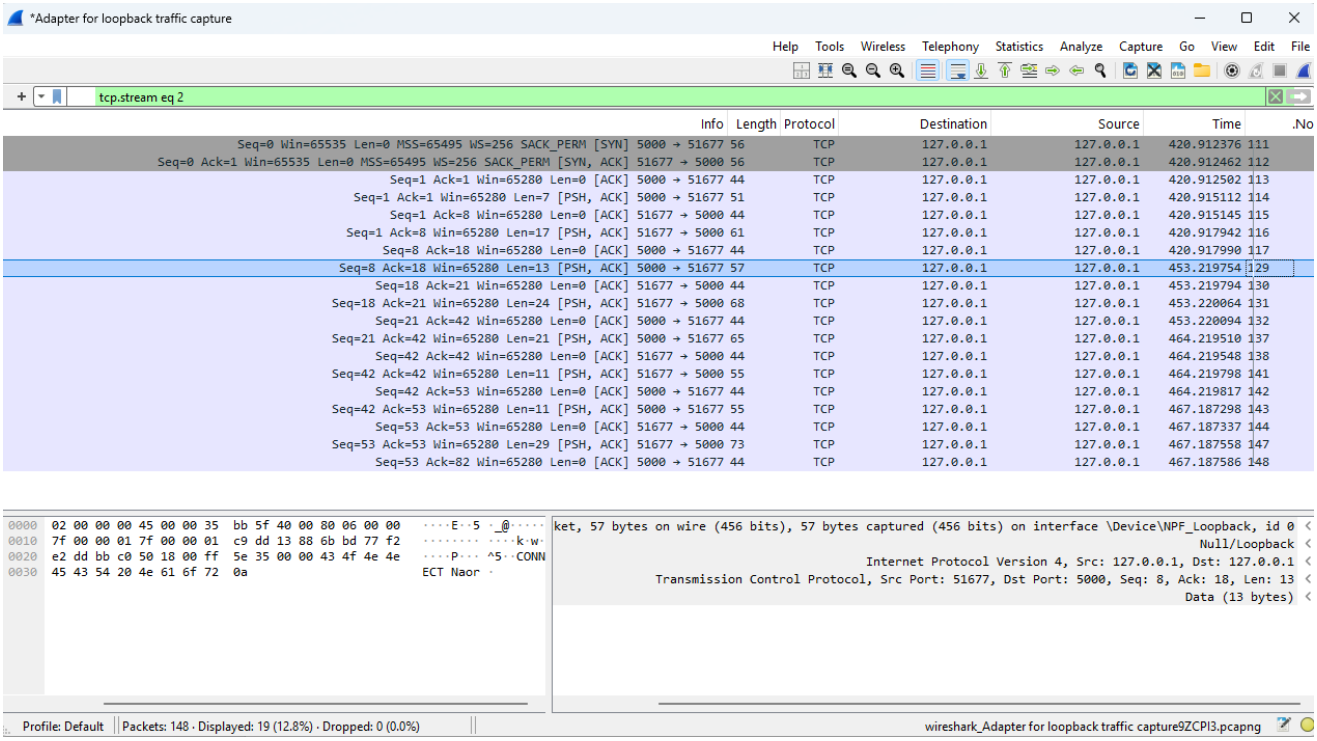


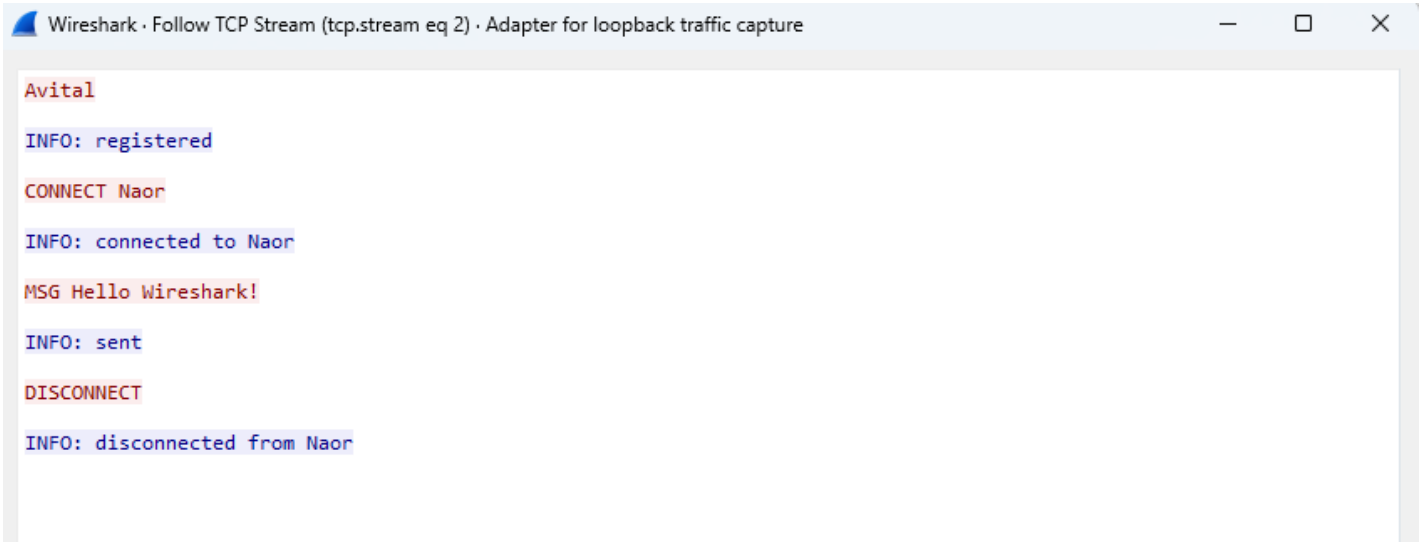
שגיאת "שם משתמש תפוס":



ניתוח תעבורה של היישום עד שכבת הרשת (Network Layer)

ביצענו לכידת תעבורה ב-Wireshark בדי לוודא שהפרוטוקול פועל כנדרש. הלכידה בוצעה בממשק ה-Loopback.





ניתוח השכבות (לפי התעבורה שנלכדה):

1. **שכבת האפליקציה:** ניתן לראות בבירור את תוכן הפרוטוקול שלנו בטקסט קריא (Plaintext): הפקודות CONNECT Naor, MSG Hello Wireshark! עוברות בדיוק כפי שהוגדרו בקוד.
2. **שכבת התעבורה (Transport - TCP):** המידע מועבר על גבי פרוטוקול TCP. בצילום ניתן לראות את ה-Stream המלא, ואת הדגלים PSH, ACK המעידים על דחיפת המידע ליישום. פורט היעד הוא 5000.
3. **שכבת הרשת (Network - IP):**
   - **כתובות IP:** מכיוון שההרצה הייתה מקומית, כתובת המקור (Source IP) וכתובת היעד (Destination IP) הן **127.0.0.1** (Loopback).
   - **פרוטוקול IP:** בשכבה זו, הפרוטוקול המזוהה הוא IPv4, והוא נושא בתוכו את סגמנט ה-TCP.
   - ניתוח זה מוכיח שהמידע עבר את כל תהליך ה-Encapsulation משכבת האפליקציה ועד לרמת ה-IP בצורה תקינה.

תיאור שימוש בבינה מלאכותית (AI)

בפרויקט זה נעזרנו בכלי AI למספר מטרות:

1. **ייעול האלגוריתמיקה:** עזרה בתכנון נכון של מנגנון ה-Multi-threading בשרת למניעת התנגשויות.
2. **שיפור ממשק המשתמש (GUI):** סיוע בכתיבת קוד ה-Tkinter ליצירת עיצוב מודרני (Dark Mode) ונוח.

דוגמאות לפרומפטים:

- "כיצד ניתן לטפל במספר לקוחות במקביל ב-Python באמצעות שימוש ב-sockets ו-threading מבלי לחסום את הריצה (non-blocking)?"

- "צור קוד לממשק גרפי (GUI) במצב כהה (Dark Mode) עבור אפליקציית צ'אט ב-Tkinter, עם צבעים נפרדים להודעות שנשלחו ולהודעות שהתקבלו."